Jason exercises: single agent

Run command: java -jar jedit.jar &
from jason-2.4/jedit

Playing with initial beliefs and rules, and basic usage of context and test goals

```
/* Initial beliefs and rules */

stored(orange, 100).
stored(lemon, 200).
stored(lime, 50).
stored(grapefruit, 400).

/* Initial goals */

!start.

/* Plans */

+!start : true <- ?stored(X, Y); .print("What is stored today? ", X, " ", Y).
```
(to show that there is no “intuitive” way to retrieve all the couples which satisfy “store”).

Add
```
stored(apple, 0).
stored(pear, 0).
stored(banana, 50).
```
and
```
available(X) :- stored(X, A) & A > 0.
```
and
```
+!start : available(X) <- .print("What is available today? ", X).
```
(to show a simple rule, and introduce how unification and resolution work)

```
+!start : stored(X, Y) <- .print("What is stored today? ", X, " ", Y); .fail.
+!start : stored(X, Y) <- .print("What is stored today? ", X, " ", Y).
```
(to show another way of testing that some fact hold, but in the context instead of as a testing goal; to show that once a plan is selected for execution, there is no automatic recovery if it fails).

```
+!start : stored(cherry, Y) <- .print("What is stored today? ", cherry, " ", Y).
+!start : stored(lemon, Y) <- .print("What is stored today? ", lemon, " ", Y).

+!start : true <-  ?stored(cherry, Y); .print("What is stored today? ", cherry, " ", Y).
+!start : true <-  ?stored(lemon, Y); .print("What is stored today? ", lemon, " ", Y).
```
(to better show the difference between making a test in the context, and making a test in the body)

```
/* Initial beliefs and rules */

stored(orange, 100).
stored(lemon, 200).
stored(lime, 50).
stored(grapefruit, 400).
isa(orange, citrus).
isa(lemon, citrus).
isa(lime, citrus).
isa(grapefruit, citrus).
stored(citrus, Fruit, X) :- stored(Fruit, X) & isa(Fruit, citrus).

/* Initial goals */

!start.

/* Plans */

+!start : stored(citrus, Fruit, Y) <- .print("What is stored today? ", Fruit, " ", Y).
```

(to show how to model and exploit ontology-like hierarchies)

```
stored(Superclass, Subclass, X) :- stored(Subclass, X) &  isa(Subclass, Superclass).
```
(to generalize the notion of stored/3)

A complex example with recursion and lists

```
/* Initial beliefs and rules */

stored(eggplant, 100).
stored(salad, 1200).
stored(tomato, 250).
stored(potato, 480).
stored(orange, 100).
stored(lemon, 200).
stored(lime, 50).
stored(grapefruit, 400).

transitive_stored(Superclass, Amount) :-
.findall(st(Subclass, X), (stored(Subclass, X) &
                           transitive_isa(Subclass, Superclass)),
                           ListWithAmount) &
.print("Stored amount of ", Superclass, ": ", ListWithAmount, "\n") &
sum(ListWithAmount, Amount).

isa(orange, citrus).
isa(lemon, citrus).
isa(lime, citrus).
isa(grapefruit, citrus).
isa(citrus, fruit).
isa(fruit, vegetable).
isa(eggplant, vegetable).
isa(potato, vegetable).
isa(tomato, vegetable).
isa(salad, vegetable).

transitive_isa(X, Y) :- isa(X, Y).
transitive_isa(X, Z) :- isa(X, Y) & transitive_isa(Y, Z).

sum([], 0).
sum([st(_, A)|T], Res+A) :- sum(T, Res).

/* Initial goals */

!start.

/* Plans */

//+!start : transitive_stored(citrus, Y) <- .print("How many citrus are stored today? ", Y).
+!start : transitive_stored(vegetable, Y) <- .print("How many vegetables are stored today? ", Y).
```